封面：Lecture 2

用线性分类器进行图像分类

Image Classification with Linear Classifiers

Fei-Fei Li & Ensan Adeli & Durante & Johnson & Xiang

2026 年 4 月 2 日

Lecture 2:

Image Classification with Linear Classifiers

Administrative: Assignment 1

Will be out **TODAY 4/2, due 4/16 by 11:59 PM**

- K-Nearest Neighbor
- Linear classifiers: Softmax
- Two-layer neural network
- Image features
- Deep neural network and optimizers

课程事务：作业 1

今天（4/2）发布，4/16 晚 11:59 截止

包含内容：

· K 近邻（K-Nearest Neighbor）

· 线性分类器：Softmax

· 两层神经网络

· 图像特征

· 深度神经网络与优化器

Administrative: Course Project

Project proposal due 4/23 (Thursday) 11:59 pm. We will release proposal details on 4/16

Contact us on Ed, each project team will eventually have a TA assigned to them for future questions after the project proposal is due.

Use this Ed thread for finding project partners. We will also later post a Google Form to find project partners.

“Is X a valid project for 231n?” --- Ed private post / TA Office Hours

More info on the website

课程事务：课程项目

· 项目提案（proposal）4/23（周四）晚 11:59 截止；提案细节将于 4/16 发布

· 通过 Ed 论坛联系我们；提案截止后，每个项目组会分配一名助教负责后续答疑

· 用 Ed 的指定帖找队友，之后也会发布 Google 表单帮忙组队

· “X 算不算 231n 的合格项目？”——请在 Ed 发私密帖或参加助教答疑时间（Office Hours）

· 更多信息见课程网站

Administrative: Discussion Sections

This Friday 12:30 pm-1:20 pm, in person at NVIDIA Auditorium, remote on Zoom (recording will be made available)

Python / Numpy, Google Colab

课程事务：讨论课（Discussion Section）

· 本周五 12:30–13:20，NVIDIA 礼堂线下进行；Zoom 同步线上（提供录像）

· 主题：Python / NumPy、Google Colab 的使用

课程大纲

深度学习基础——感知与理解视觉世界：

数据驱动方法、线性分类、K 近邻、损失函数、优化、反向传播、多层感知机、神经网络、激活函数、数据增强、迁移学习、优化器、卷积、PyTorch、RNN/注意力/Transformer、归一化层、架构设计

重构与交互视觉世界：

视频理解、视觉与语言、3D 视觉、目标检测与分割、风格迁移、生成模型、自监督学习、图像生成、机器人与具身智能、以人为中心的 AI、公平与伦理

▸ 图：课程知识地图（本讲属于“深度学习基础”部分）

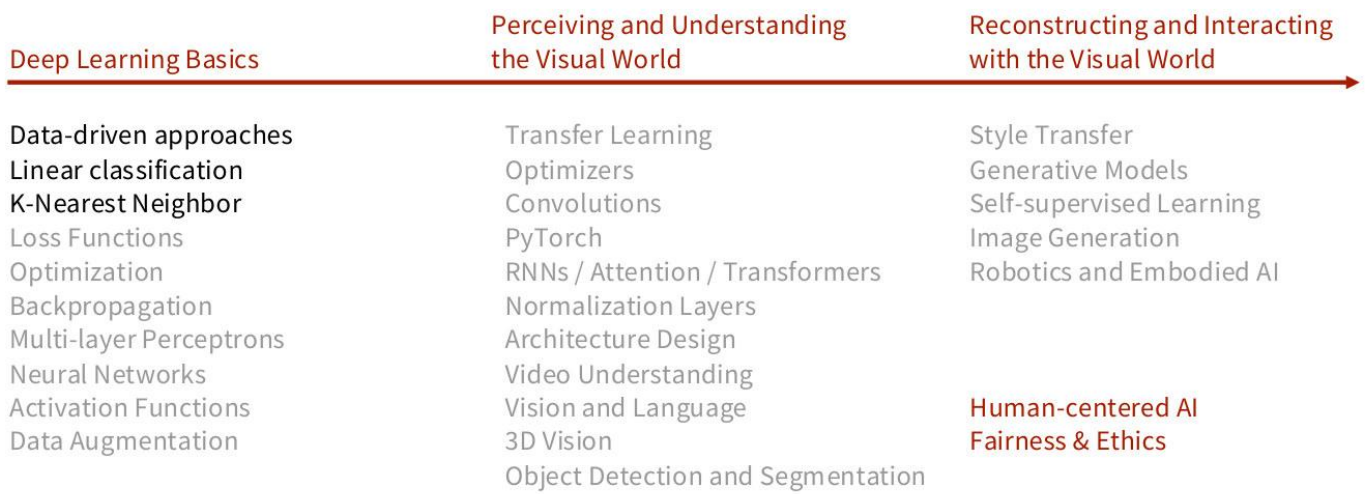


Image Classification

A Core Task in Computer Vision

Today:

- The image classification task
- Two basic data-driven approaches to image classification
  - K-nearest neighbor and linear classifier

图像分类：计算机视觉的核心任务

今天的内容：

· 图像分类任务是什么

· 两种基本的数据驱动图像分类方法：

- K 近邻（K-Nearest Neighbor）
- 线性分类器（Linear Classifier）

图像分类：CV 的核心任务

任务：给定一张图片，从固定的标签集合中
{狗, 猫, 卡车, 飞机, ...}
选出正确的类别 → 输出“猫”

- 图：一张猫的照片，分类器给出标签 cat

Image Classification: A core task in Computer Vision



This image by Nikita is licensed under CC BY 2.0

(assume given a set of possible labels)
{dog, cat, truck, plane, ...}

→ cat

问题：语义鸿沟（Semantic Gap）

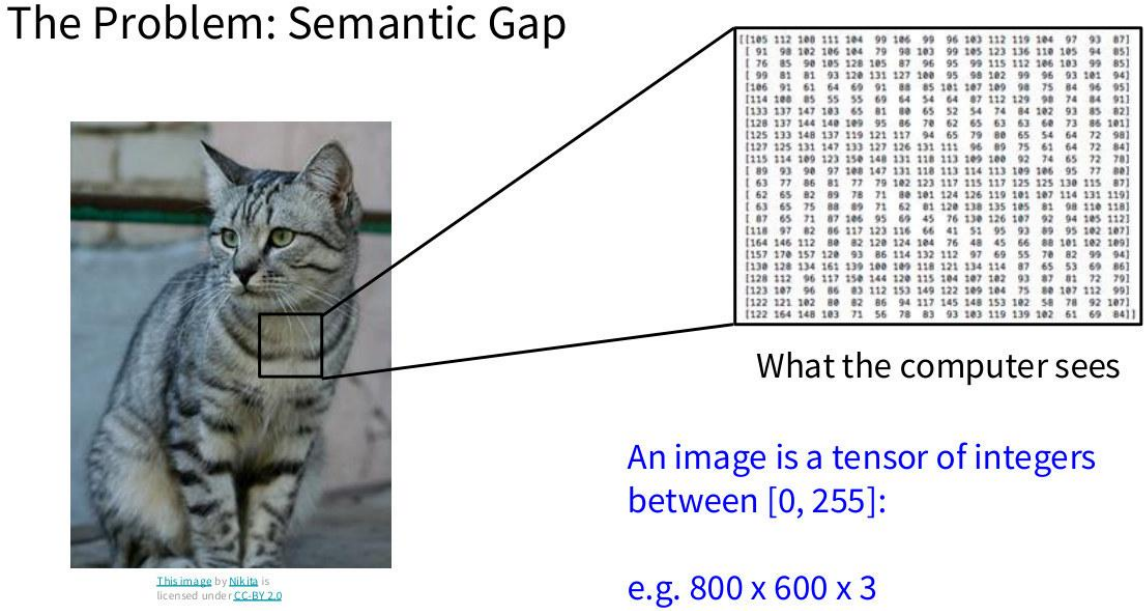
计算机“看到”的是什么?

一张图像就是一个取值在 [0, 255] 之间的整数张量（tensor），

例如 800 × 600 × 3（RGB 三个颜色通道）

人类看到“一只猫”，计算机只看到一堆数字——

如何从数字跨越到“语义”，就是分类要解决的核心难题



挑战 1：视角变化（Viewpoint）

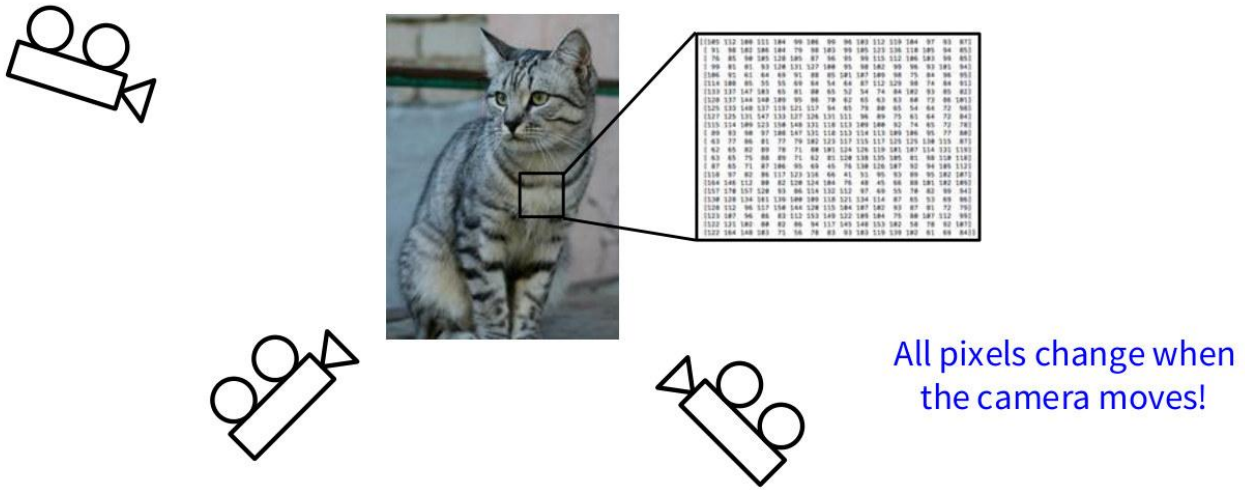
相机一动，所有像素都变了！

同一只猫从不同角度拍摄，像素值完全不同，

但语义（标签“猫”）不变

- 图：同一场景不同视角的照片

Challenges: Viewpoint variation



All pixels change when the camera moves!

This image by Nikita is licensed under CC-BY 2.0

挑战 2：光照（Illumination）

同一物体在不同光照条件下，
像素值差别巨大

▸ 图：同一场景在明暗不同光线下的多张照片

Challenges: Illumination



This image is CC0 1.0 public domain
This image is CC0 1.0 public domain
This image is CC0 1.0 public domain
This image is CC0 1.0 public domain

挑战 3：背景杂乱（Background Clutter）

物体常常淹没在复杂背景中，
难以与周围环境区分

- 图：隐藏在杂乱背景中的目标物体

Challenges: Background Clutter



This image is CC0 1.0 public domain



This image is CC0 1.0 public domain

挑战 4：遮挡（Occlusion）

目标物体经常只露出一部分，
被其他物体挡住

- 图：被遮挡的动物照片

Challenges: Occlusion



This image is CC0 1.0 public domain



This image is CC0 1.0 public domain



This image by jonsson is licensed under CC-BY 2.0

挑战 5：形变（Deformation）

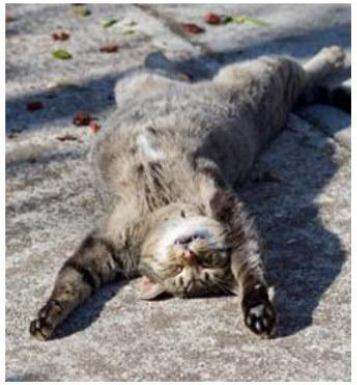
物体是柔软可变形的，
可以呈现各种姿态（猫可以蜷成一团、拉长身体……）

▸ 图：各种姿势的猫

Challenges: Deformation



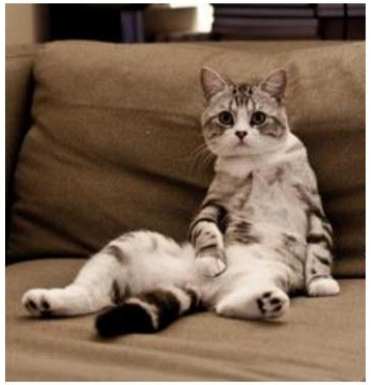
This image by Umberto Salvagnin is licensed under CC-BY 2.0



This image by Umberto Salvagnin is licensed under CC-BY 2.0



This image by sarebear is licensed under CC-BY 2.0



This image by Tom Thai is licensed under CC-BY 2.0

Challenges: Intraclass variation



This image is CC0 1.0 public domain

挑战 6：类内差异（Intraclass Variation）

同一类别的不同个体之间

外观差异可以非常大

- 图：不同品种、毛色的猫

挑战 7：上下文（Context）

物体的含义往往依赖上下文环境

- 图：一个说明“语境影响识别”的梗图

Challenges: Context

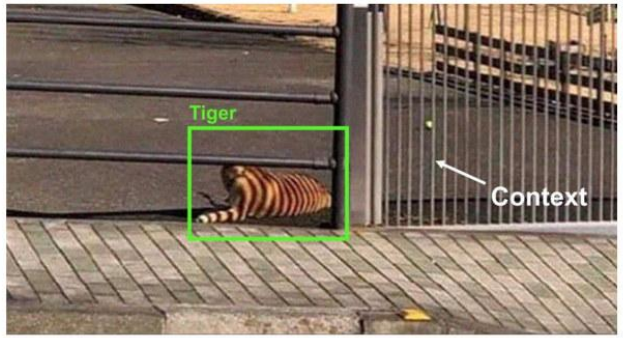




Image source: https://www.linkedin.com/posts/ralph-aboujaoude-diaz-40838313_technology-artificialintelligence-computervision-activity-6912446088364875776-h-lq?utm_source=linkedin_share&utm_medium=member_desktop_web

现代计算机视觉算法

▸ 图：由数百万张小图片拼贴成的 ImageNet 大图

ImageNet 是推动现代计算机视觉发展的
大规模图像数据集

Modern computer vision algorithms



This image is CC0 1.0 public domain

一个图像分类器

与“给数字列表排序”不同——
识别猫（或其他类别）
没有显而易见的硬编码（hard-code）算法可以写出来

→ 需要数据驱动的方法

An image classifier

```
def classify_image(image):
  # Some magic here?
  return class_label
```

Unlike e.g. sorting a list of numbers,

no obvious way to hard-code the algorithm for recognizing a cat, or other classes.

前人做过尝试

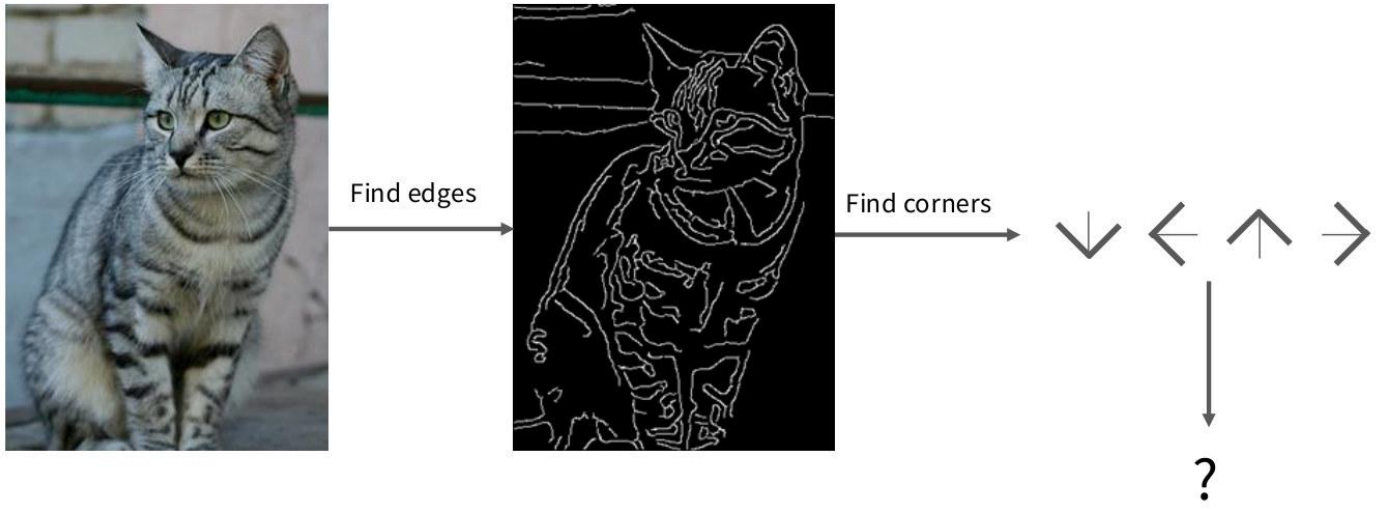
经典流程：找边缘（Find edges）→ 找角点（Find corners）→ ?

- 图：猫照片 → Canny 边缘图 → 一些角点符号 → 一个问号

即使提取了边缘和角点，
仍然无法直接得到“这是一只猫”

参考：John Canny, “A Computational Approach to Edge Detection”, IEEE TPAMI 1986

Attempts have been made



John Canny, "A Computational Approach to Edge Detection", IEEE TPAMI 1986

机器学习：数据驱动方法（Data-Driven Approach）

三步走：

1. 收集带标签的图像数据集
2. 用机器学习算法训练一个分类器
3. 在新图像上评估分类器

▸ 图：示例训练集（猫/狗等多类图片及其标签）

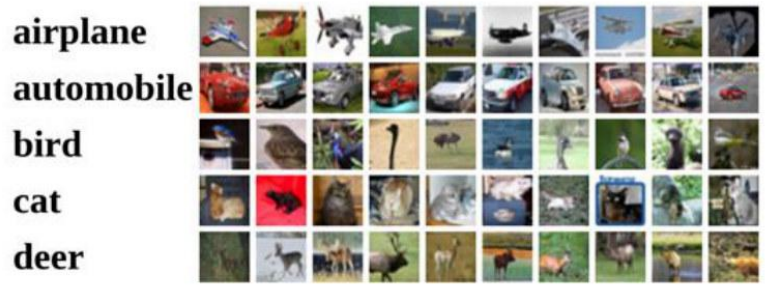
Machine Learning: Data-Driven Approach

1. Collect a dataset of images and labels
2. Use Machine Learning algorithms to train a classifier
3. Evaluate the classifier on new images

```
def train(images, labels):
  # Machine learning!
  return model
```

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

Example training set



最近邻分类器（Nearest Neighbor）

- 图：CIFAR-10 数据集的图片网格

下面开始介绍第一个具体的分类器

Nearest Neighbor Classifier

第一个分类器：最近邻

训练：记住所有数据和标签

预测：找出与测试图最相似的训练图像，
直接预测它的标签

▸ 图：Memorize all data and labels → Predict the label of the most similar training image

First classifier: Nearest Neighbor

```
def train(images, labels):
  # Machine learning!
  return model
```

→ Memorize all data and labels

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

→ Predict the label of the most similar training image

最近邻：距离度量

要素：

· 带标签的训练数据

· 查询数据（要分类的新图像）

· 距离度量（用来比较图像相似度）

▸ 图：训练集里有鹿/鸟/飞机/猫/汽车等类别，查询图旁打着问号

First classifier: Nearest Neighbor



Training data with labels



query data

Distance Metric $\left| \quad , \quad \right| \rightarrow \mathbb{R}$

用距离度量比较图像：L1 距离

L1 距离（曼哈顿距离）：

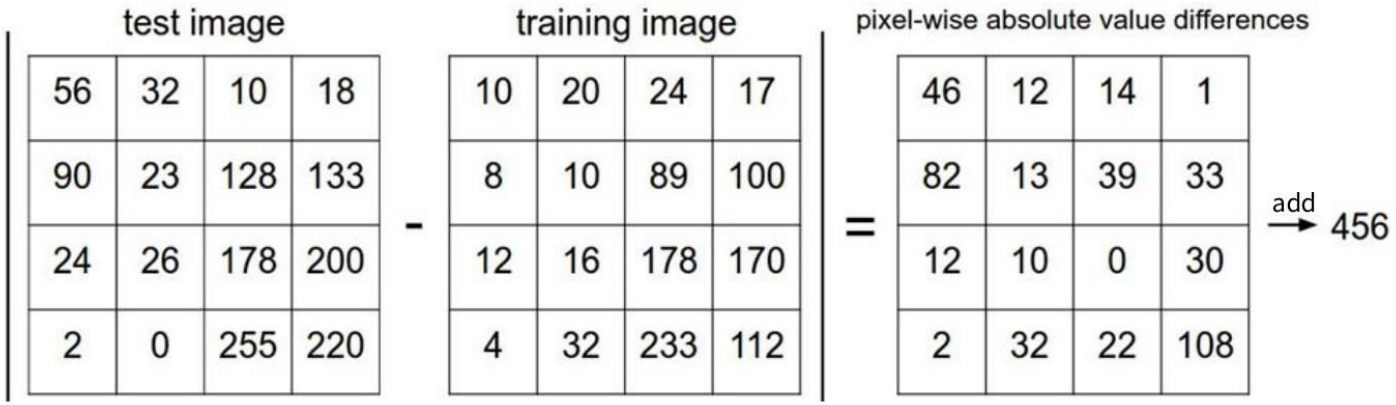
把两张图逐像素相减取绝对值，再全部加起来

d1(I1, I2) = Σ |I1 的像素 − I2 的像素|

▸ 图：两张 4×4 小图逐格相加（add）的过程

Distance Metric to compare images

L1 distance: $d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$



最近邻分类器（代码）

Python / NumPy 实现（左侧代码）：

· train(X, y)：简单记住训练数据
  self.Xtr = X; self.ytr = y
· predict(X)：对每张测试图
  - 用 L1 距离计算它与所有训练图的距离：
    distances = np.sum(np.abs(self.Xtr - X[i,:]), axis=1)
  - 取距离最小的索引：min_index = np.argmin(distances)
  - 预测该最近样本的标签：Ypred[i] = self.ytr[min_index]

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

Nearest Neighbor classifier

最近邻分类器（图示：训练）

▸ 图：把训练数据（图片+标签）存起来

训练阶段 = 记住训练数据

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

Nearest Neighbor classifier

Memorize training data

最近邻分类器（图示：预测）

对每张测试图像：

· 找到最接近的训练图像

· 用最近图像的标签作为预测结果

▸ 图：测试图 → 与训练集逐个比较 → 取最近者

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

Nearest Neighbor classifier

For each test image:
Find closest train image
Predict label of nearest image

最近邻：训练/预测的速度

问：有 N 个样本时，训练和预测分别多快？

答：训练 O(1)，预测 O(N)

这很糟糕！我们想要的是：
· 预测快
· 训练慢一点没关系

（最近邻恰好相反）

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

Nearest Neighbor classifier

Q: With N examples, how fast are training and prediction?

Ans: Train O(1),
predict O(N)

This is bad: we want classifiers that are fast at prediction; slow for training is ok

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

Nearest Neighbor classifier

Many methods exist for fast / approximate nearest neighbor (beyond the scope of 231N!)

A good implementation:
https://github.com/facebookresearch/faiss

Johnson et al, "Billion-scale similarity search with GPUs", arXiv 2017

快速/近似最近邻

存在很多快速或近似最近邻的方法
（超出本课范围！）

一个优秀的开源实现：
https://github.com/facebookresearch/faiss

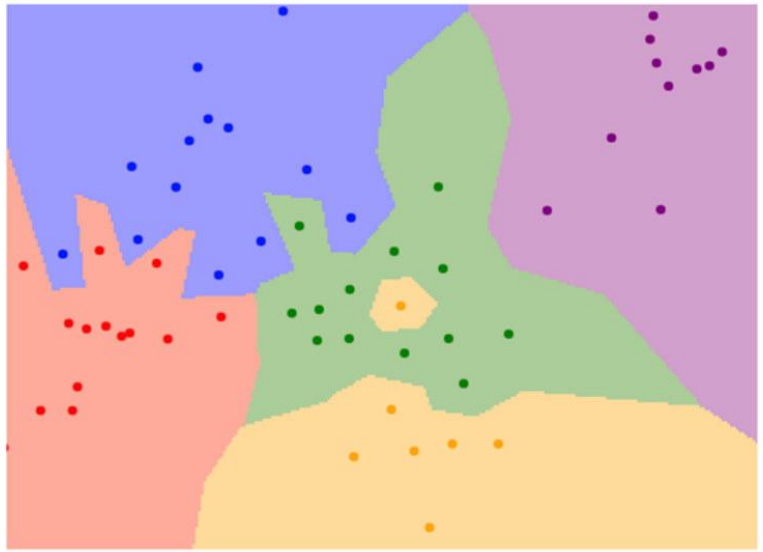
参考：Johnson et al., "Billion-scale similarity search with GPUs", arXiv 2017

这看起来像什么？ (1-NN 决策区域)

▸ 图：二维平面上 1-最近邻的决策区域图
不同颜色的区域代表预测为不同类别，
散点代表训练样本

每个点都被分给它最近的训练样本所属的类，
因此决策边界非常不规则（容易过拟合）

What does this look like?



1-nearest neighbor

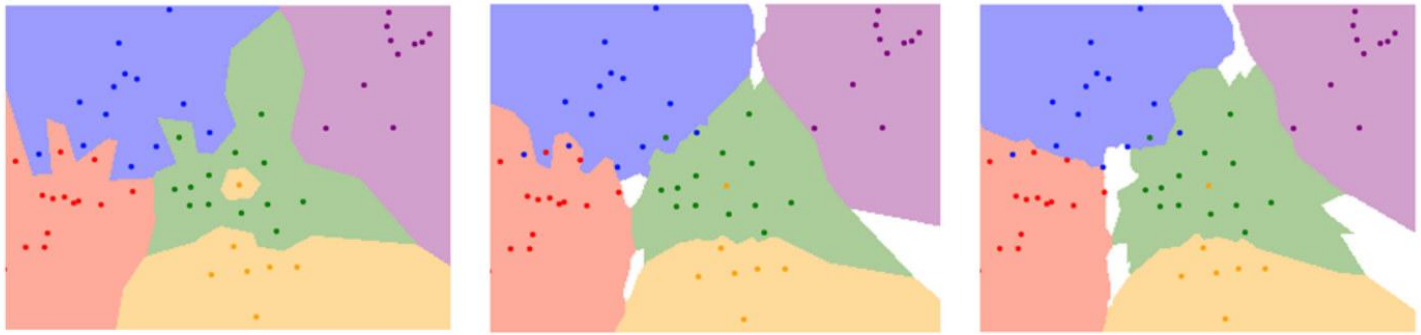
K 近邻（K-Nearest Neighbors）

改进：不再只复制最近 1 个邻居的标签，
而是从最近的 K 个点中做多数投票（majority vote）

▸ 图：同一份数据在 K=1、K=3、K=5 下的决策区域对比，K 越大边界越平滑

K-Nearest Neighbors

Instead of copying label from nearest neighbor,
take majority vote from K closest points

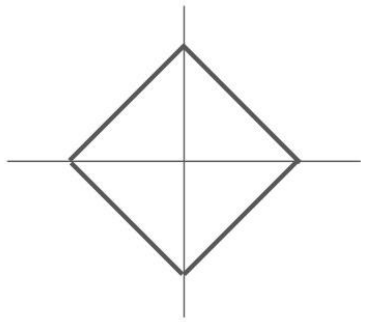


K = 1　　K = 3　　K = 5

K-Nearest Neighbors: Distance Metric

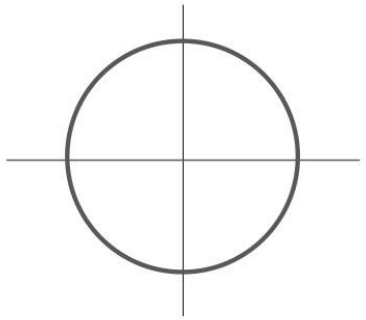
L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K 近邻：距离度量

两种常用距离：

· L1（曼哈顿）距离：沿坐标轴（网格线）累加绝对差

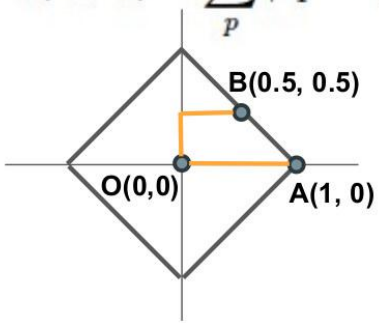
· L2（欧氏）距离：直线距离

▸ 图：同一数据、K=1 时，L1 与 L2 给出的决策边界形状不同

K-Nearest Neighbors: Distance Metric - Example

L1 Distance: Measures distance by moving along grid lines (like walking in a city with square blocks).

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



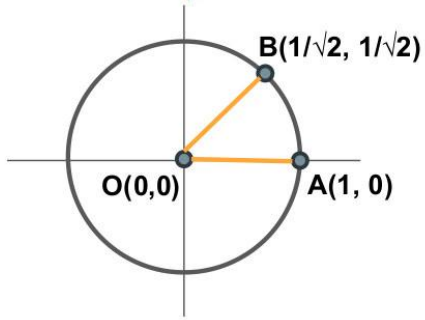
d_1(O,A) = |0-1| + |0-0| = 1

d_1(O,B) = |0-0.5| + |0-0.5| = 0.5 + 0.5 = 1

d_1(O,A) = d_1(O,B) = 1

L2 Distance: Measures the straight-line distance (as the crow flies).

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



d_2(O,A) = sqrt((0-1)² + (0-0)²) = sqrt(1²) = 1

d_2(O,B) = sqrt((0-1/√2)² + (0-1/√2)²) = sqrt(1/2+1/2) = sqrt(1) = 1

d_2(O,A) = d_2(O,A) = *1*

距离度量示例

O(0,0)、A(1,0)、B(0.5, 0.5)：

L1 距离：

d1(O,A) = |0-1| + |0-0| = 1

d1(O,B) = |0-0.5| + |0-0.5| = 1

→ d1(O,A) = d1(O,B)

L2 距离（B 取 (1/√2, 1/√2)）：

d2(O,A) = sqrt(1² + 0²) = 1

d2(O,B) = sqrt((1/√2)² + (1/√2)²) = 1

→ d2(O,A) = d2(O,B)

直观理解：

· L1：像沿方格街区走路（走网格线）

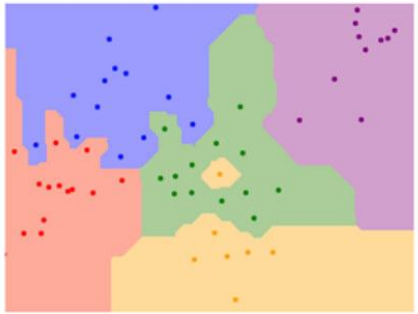
· L2：直线距离（乌鸦飞行路线）

坐标系不同时，两种度量的结果会不同

K-Nearest Neighbors: Distance Metric

L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



K = 1

L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K = 1

距离度量影响决策边界

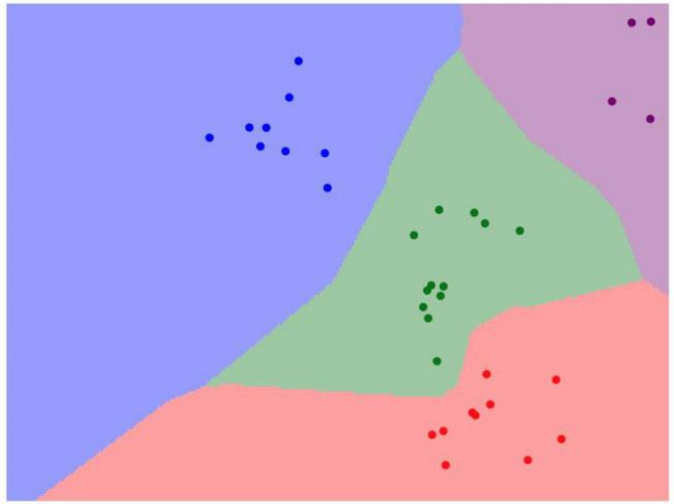
- 图：相同数据、K=1 下，L1（曼哈顿）距离与 L2（欧氏）距离分别得到的决策边界

选择不同的距离度量，
模型的决策方式随之改变

K 近邻：自己动手试试！

在线交互演示（可调 K 和距离度量）：
http://vision.stanford.edu/teaching/cs231n-demos/knn/

K-Nearest Neighbors: try it yourself!



http://vision.stanford.edu/teaching/cs231n-demos/knn/

Hyperparameters

What is the best value of k to use?
What is the best distance to use?

These are hyperparameters: choices about the algorithms themselves.

Very problem/dataset-dependent.
Must try them all out and see what works best.

超参数（Hyperparameters）

问题：

· k 取多少最好？

· 用哪种距离最好？

这些是超参数：关于算法本身的选择（不是从数据中学到的）

· 非常依赖具体问题/数据集

· 必须全部尝试，看哪种效果最好

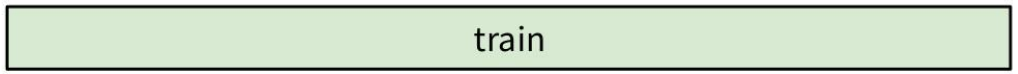
设定超参数：想法 1

想法 1：选择在【训练数据】上表现最好的超参数

- 图：只有 train（训练集）

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the training data



设定超参数：想法 1 的问题

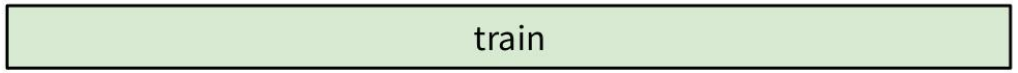
想法 1：选择在训练数据上表现最好的超参数

× 很糟：K = 1 时，在训练数据上永远完美
（每个点最近的邻居就是它自己）

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the training data

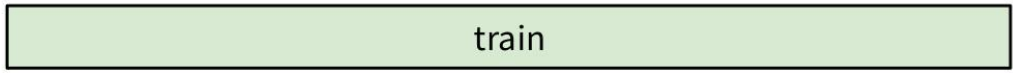
BAD: K = 1 always works perfectly on training data



Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the training data

BAD: K = 1 always works perfectly on training data



Idea #2: choose hyperparameters that work best on test data



设定超参数：想法 2

想法 1：× K=1 在训练数据上总是完美

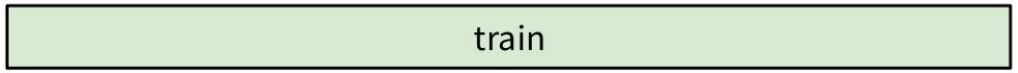
想法 2：选择在【测试数据】上表现最好的超参数

- 图：train + test 两个数据块

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the training data

BAD: K = 1 always works perfectly on training data



Idea #2: choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



Never do this!

设定超参数：想法 2 的问题

想法 2：选在测试数据上表现最好的超参数

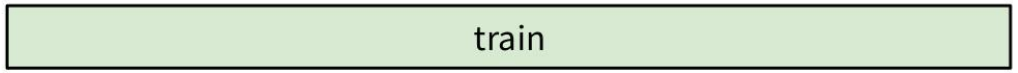
× 很糟：完全无法知道算法
在新数据上表现如何

【千万不要这么做！】
测试集只能用一次，相当于模拟“未来真实数据”

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the training data

BAD: K = 1 always works perfectly on training data



Idea #2: choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



Idea #3: Split data into train, val; choose hyperparameters on val and evaluate on test

Better!



设定超参数：想法 3（验证集）

想法 3：把数据分成 训练集 / 验证集 / 测试集；
用验证集选超参数，最后在测试集上评估

√ 更好！

▸ 图：train | validation | test 三块数据

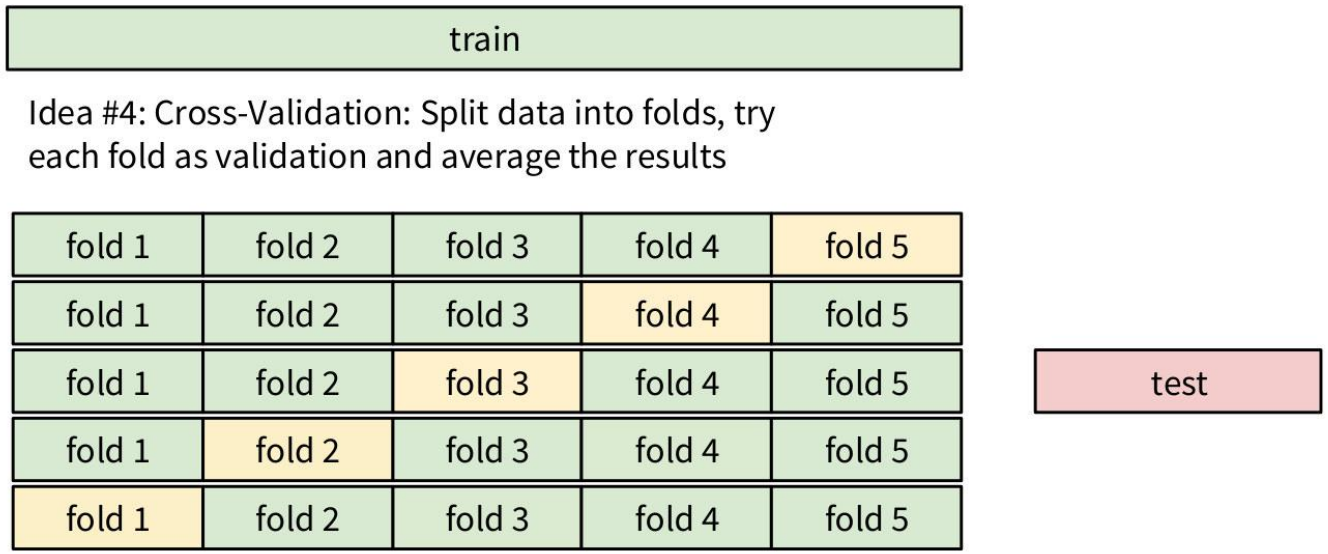
设定超参数：想法 4（交叉验证）

想法 4：交叉验证（Cross-Validation）
把数据分成若干折（fold），
轮流把每一折当验证集，其余当训练集，
最后把结果取平均

▸ 图：5 折交叉验证示意图

适用于小数据集；
深度学习中不常用（训练代价太高）

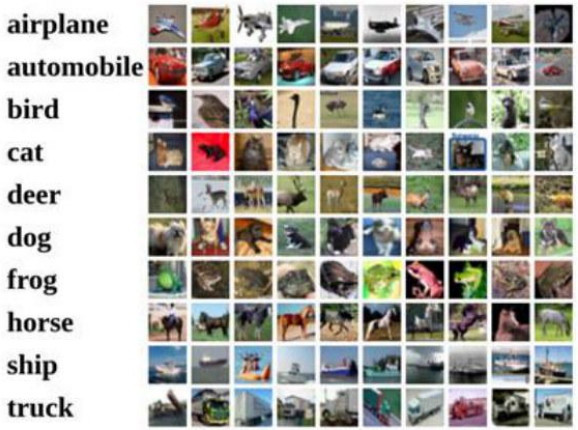
Setting Hyperparameters



Useful for small datasets, but not used too frequently in deep learning

Example Dataset: CIFAR10

10 classes
50,000 training images
10,000 testing images



Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images", Technical Report, 2009.

示例数据集：CIFAR10

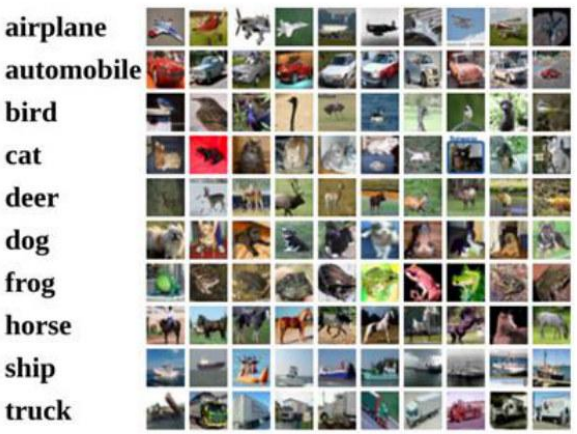
Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images", Technical Report, 2009

· 10 个类别
· 50,000 张训练图像
· 10,000 张测试图像

▸ 图：每类若干张 32×32 小图

Example Dataset: CIFAR10

10 classes
50,000 training images
10,000 testing images



Test images and nearest neighbors



Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images", Technical Report, 2009.

CIFAR10：测试图与最近邻

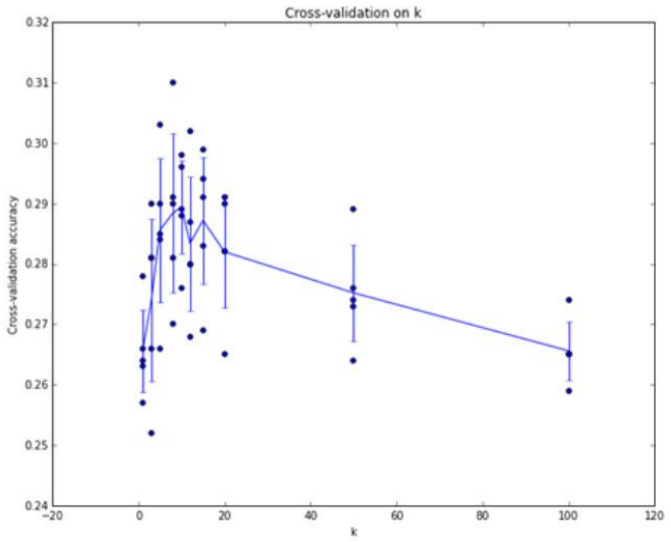
- 图：左边一列是测试图像，
右边是按像素距离找到的最近邻训练图像

可以看到：像素上“最接近”的图
在语义上经常完全不是同一类
（比如找“猫”却找回了“卡车”）

用交叉验证选 k 的实例

- 图：对 k 的取值做 5 折交叉验证的结果曲线
- 每个点 = 单次验证的结果
- 折线穿过各点的均值，误差条表示标准差
- 结论：对本数据，k ≈ 7 效果最好

Setting Hyperparameters



Example of
5-fold cross-validation
for the value of k.

Each point: single
outcome.

The line goes
through the mean, bars
indicated standard
deviation

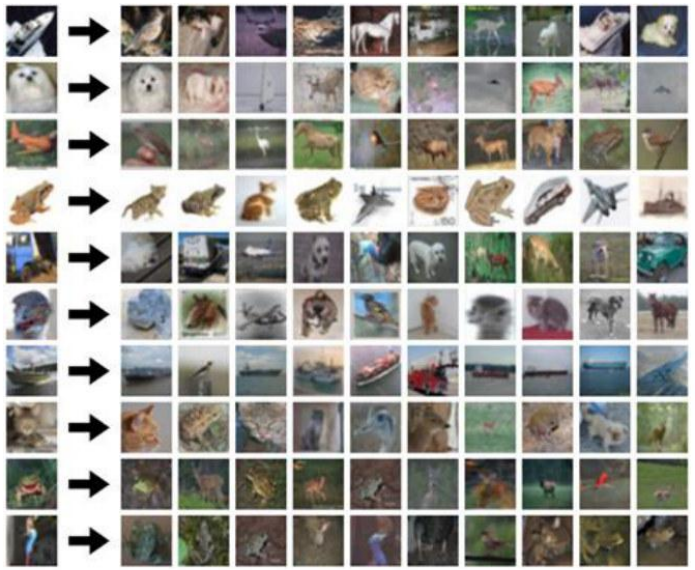
(Seems that k ~= 7 works best
for this data)

这看起来像什么？ (CIFAR-10 上的 1-NN)

- 图：每行左边一张 CIFAR-10 测试图，
右边是按 L2 像素距离找到的 10 张最近训练图

大多数“最近邻”与测试图并不同类——
说明像素距离捕捉不到语义

What does this look like?



What does this look like?



同上（标注对错）

- 图：与上页相同，但每个最近邻
- 标绿框 = 与测试图同类（找对了）
- 标红框 = 不同类（找错了）

错误远多于正确

k-Nearest Neighbor with pixel distance is never used.

- Distance metrics on pixels are not informative



(All three images on the right have the same pixel distances to the one on the left)

基于像素距离的 kNN 从不使用

基于像素距离的 k-近邻在实际中从不使用：
像素上的距离度量没有信息量（不反映语义）

▸ 图：原图 | 遮挡版 | 平移 1 像素版 | 调色版
右侧三张图与左侧原图的像素距离完全相同，
但它们在语义上差别巨大

K-Nearest Neighbors: Summary

In image classification we start with a training set of images and labels, and must predict labels on the test set

The K-Nearest Neighbors classifier predicts labels based on the **K nearest training examples**

Distance metric and K are hyperparameters

Choose hyperparameters using the validation set

Only run on the test set once at the very end!

K 近邻：小结

· 图像分类：从带标签的训练集出发，在测试集上预测标签
· KNN 分类器：基于 K 个最近的训练样本来预测
· 距离度量和 K 都是超参数
· 用验证集来选择超参数
· 测试集只在最后用一次！

线性分类器

接下来：另一种基本的数据驱动方法

Linear Classifier

参数化方法（Parametric Approach）

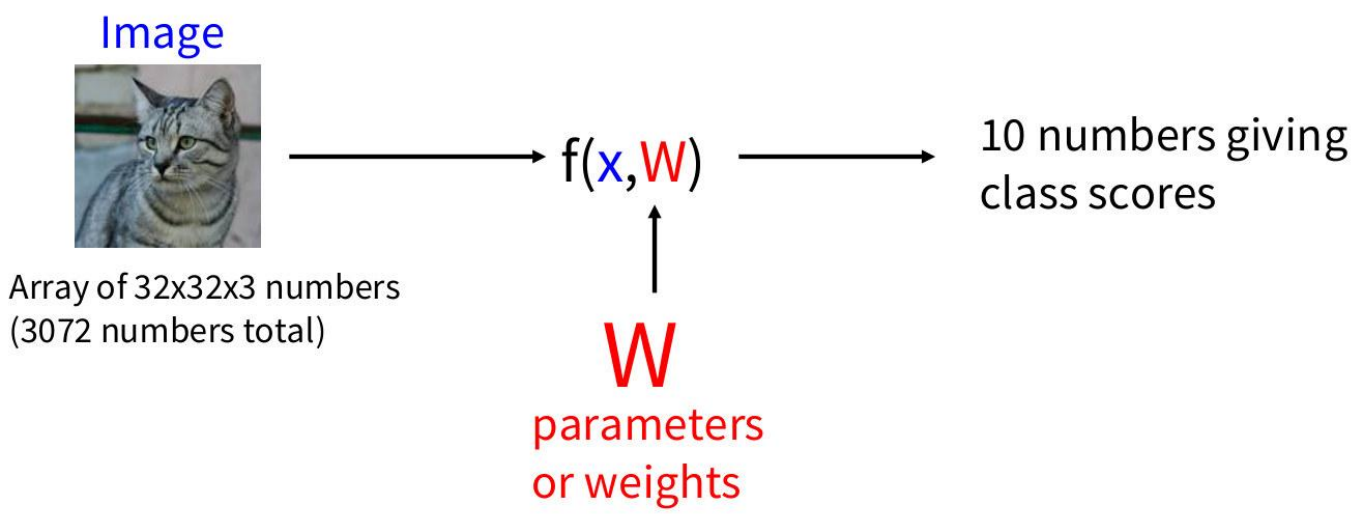
输入：图像（32×32×3 的数字数组，共 3072 个数）

f(x, W)

输出：10 个数 = 10 个类别的得分

W：参数（parameters）或权重（weights）

▸ 图：图像 → 函数 f(x,W) → 类别得分；
一次性学习所有训练数据的知识，浓缩进 W

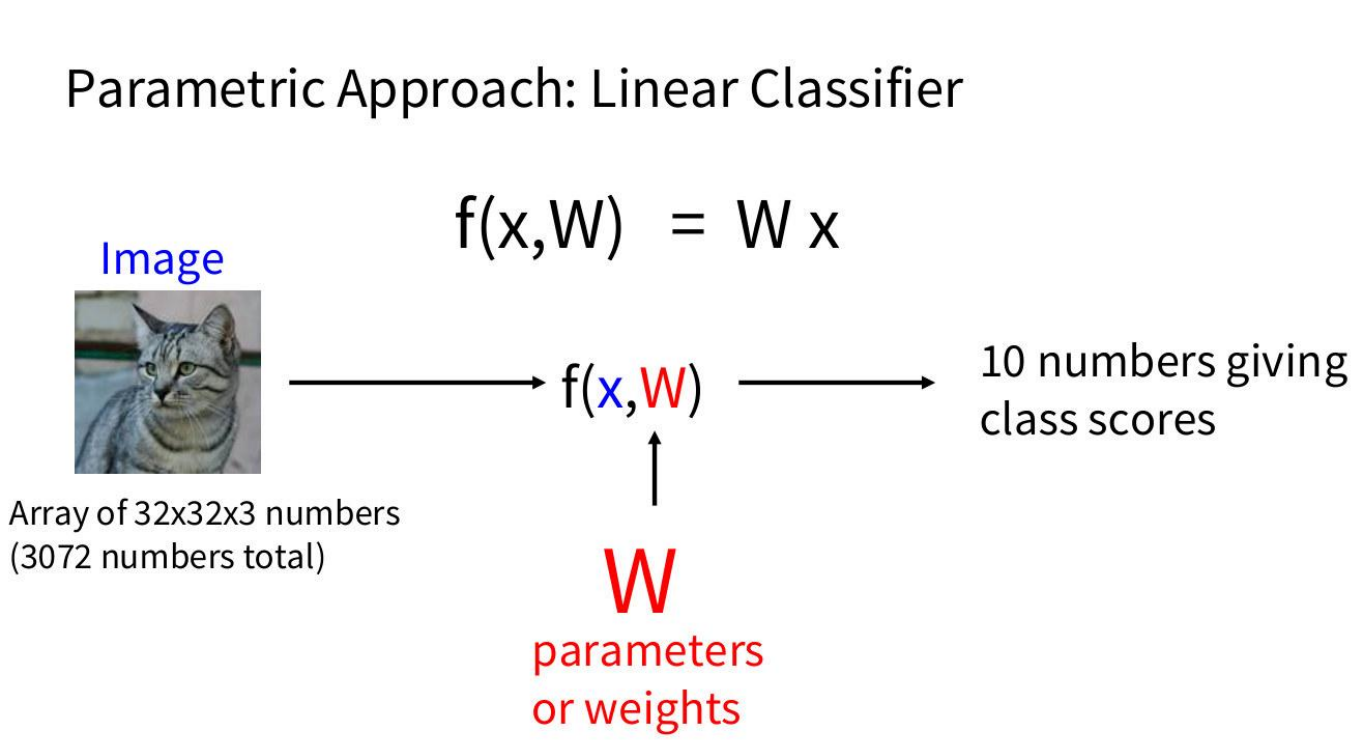


参数化方法：线性分类器

最简单的参数化方法：线性分类器

$f(x, W) = W x$

- 图：图像（3072 维向量）→ 矩阵乘法 → 10 个类别得分

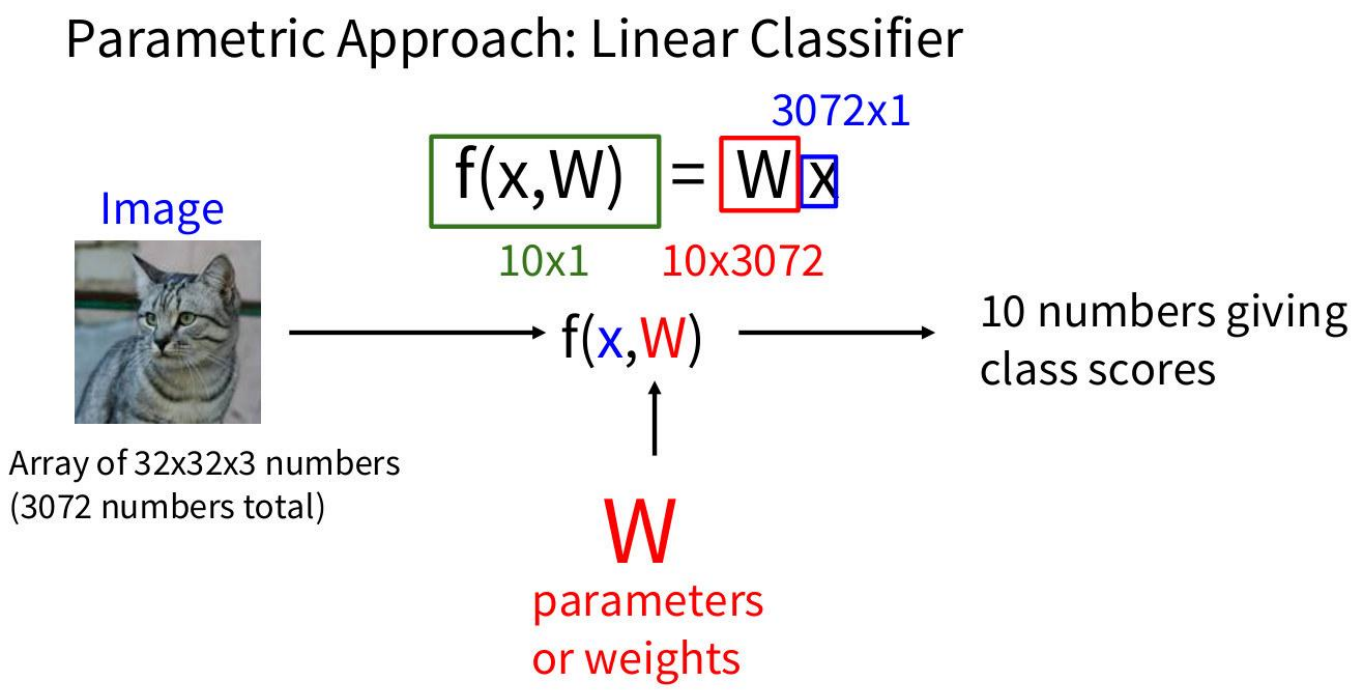


线性分类器：维度

f(x, W) = W x

· x：3072 × 1（图像展平成向量）
· W：10 × 3072（权重矩阵）
· 输出：10 × 1（每类一个得分）

参数量与训练数据大小无关——
所有知识都压缩在 W 里

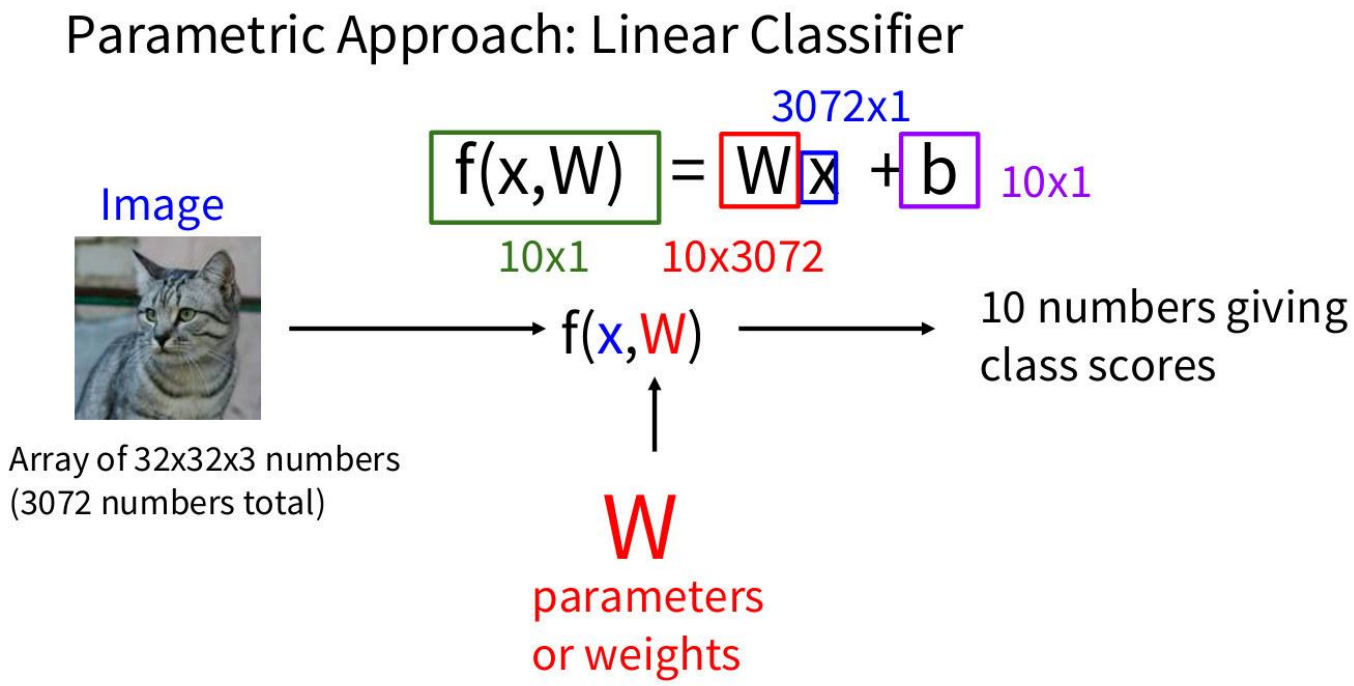


线性分类器：加偏置项

f(x, W) = W x + b

· W：10 × 3072
· x：3072 × 1
· b：10 × 1（偏置，bias）

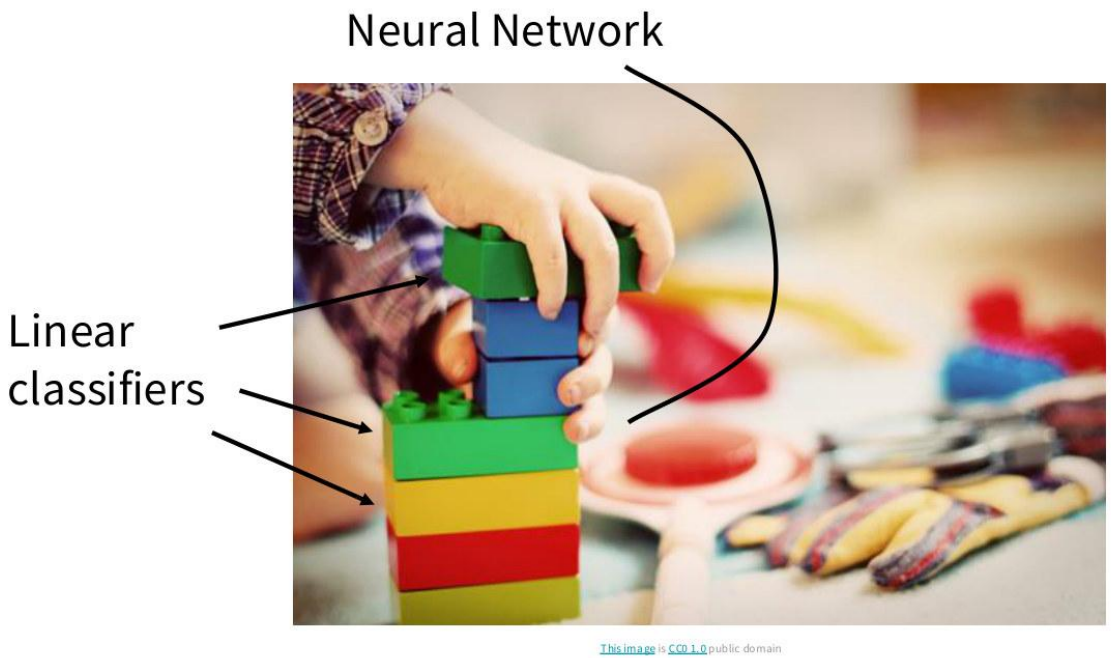
偏置不与输入相乘，
允许各类别有一个独立的“基础得分”



神经网络 = 线性分类器的堆叠

- 图：小孩搭乐高积木的照片，
  “线性分类器”指向单块积木，
  “神经网络”指向搭好的整体结构

比喻：线性分类器是积木块，
神经网络就是把它们堆叠起来的结构



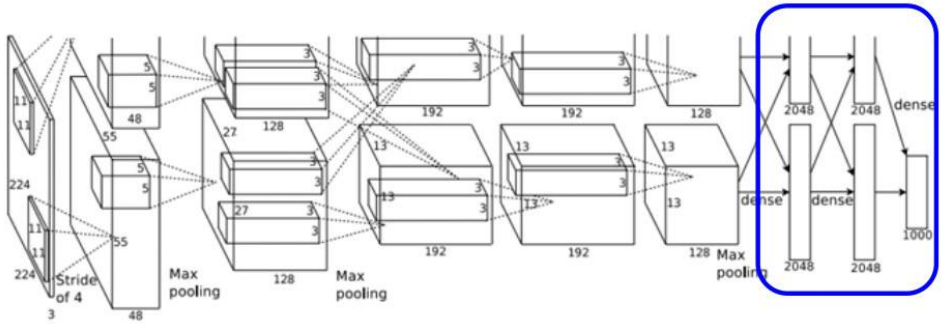
This image is CC0 1.0 public domain

线性层 → 深度网络

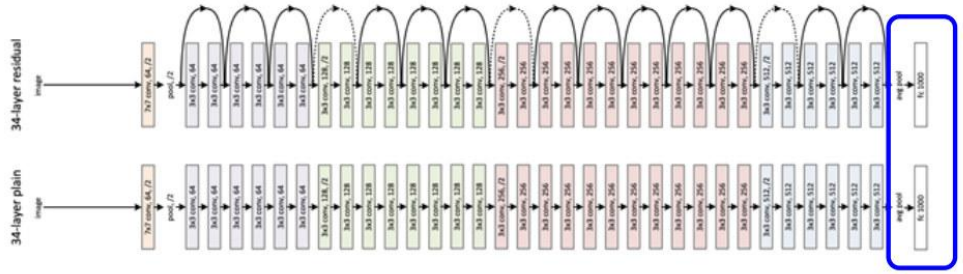
▸ 图：AlexNet（2012）和 ResNet（2015）的网络结构图

深度网络就是把很多线性层
（夹杂非线性激活）堆叠起来

参考：Krizhevsky et al. 2012；He et al. 2015



[Krizhevsky et al. 2012]



[He et al. 2015]

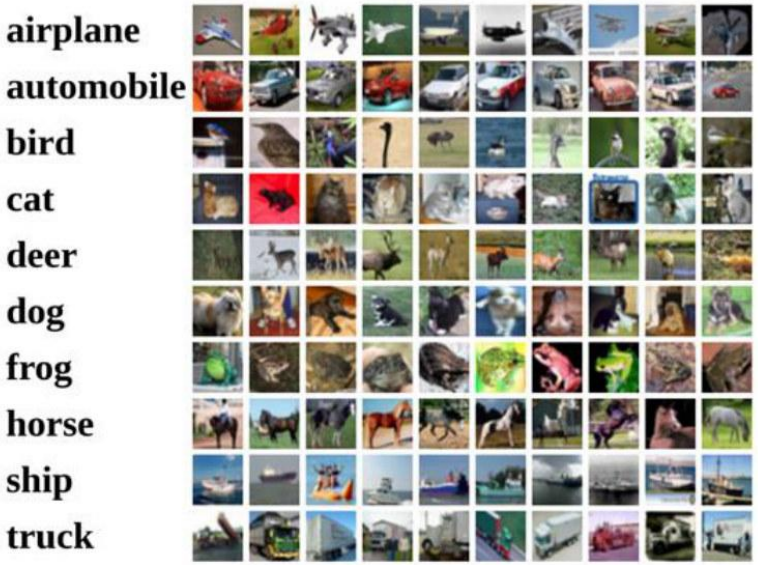
Linear layers

回顾 CIFAR10

· 50,000 张训练图像，每张 32×32×3
· 10,000 张测试图像

下面用一个更小的例子说明线性分类器

Recall CIFAR10



50,000 training images
each image is 32x32x3

10,000 test images.

小例子：4 像素图像、3 个类别

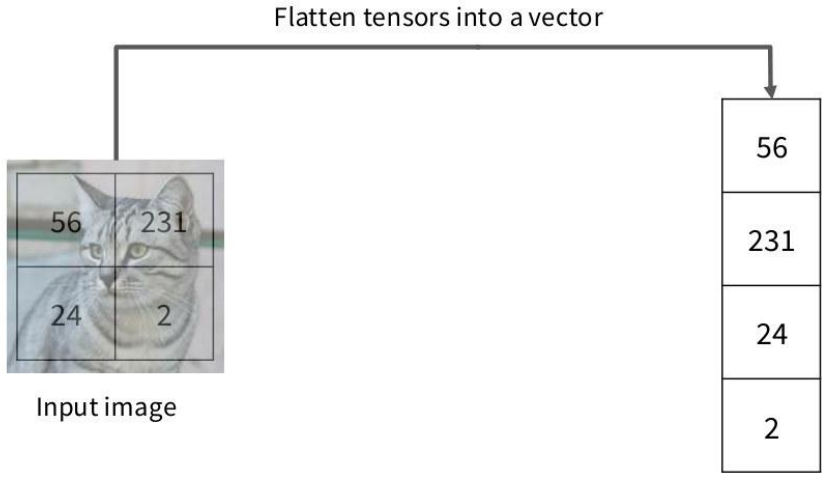
例：一张 2×2 的小图（4 个像素），3 个类别（猫/狗/船）

第一步：把张量展平（flatten）成向量

图像 [[56, 231], [24, 2]] → x = [56, 231, 24, 2]ᵀ

▸ 图：输入图像 → 展平成 4×1 列向量

Example with an image with 4 pixels, and 3 classes (cat/dog/ship)



代数视角（Algebraic Viewpoint）

W（3×4 矩阵）：

[0.2 -0.5 0.1 2.0] ← 猫 的权重行
[1.5 1.3 2.1 0.0] ← 狗 的权重行
[0 0.25 0.2 -0.3] ← 船 的权重行

x = [56, 231, 24, 2]ᵀ，b = [1.1, 3.2, -1.2]ᵀ

W x + b = 类别得分：

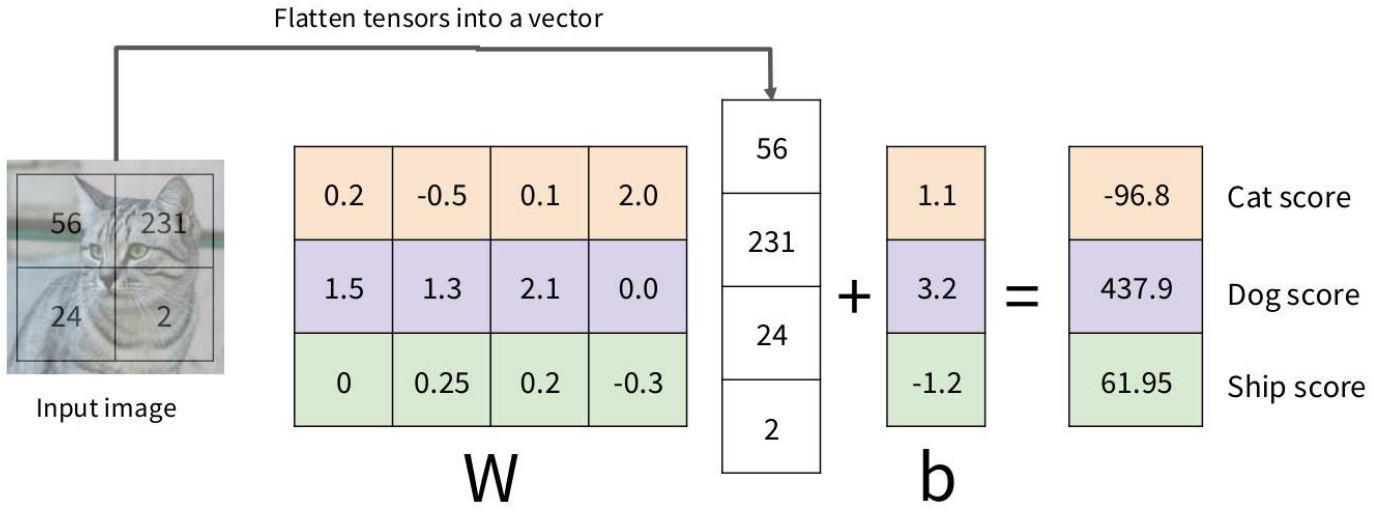
猫 = -96.8　狗 = 437.9　船 = 61.95

当前会预测“狗”——但这其实是猫的图（预测错误）

- 图：矩阵乘法的完整演算过程

Example with an image with 4 pixels, and 3 classes (cat/dog/ship)

Algebraic Viewpoint



解读线性分类器

三种理解线性分类器的视角：

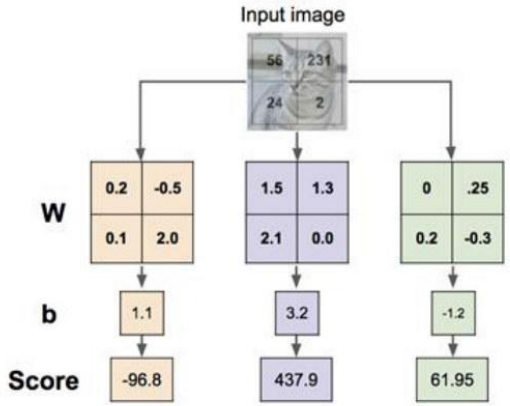
· 代数视角（上两页）

· 视觉视角（下一页）

· 几何视角（再下一页）

Interpreting a Linear Classifier

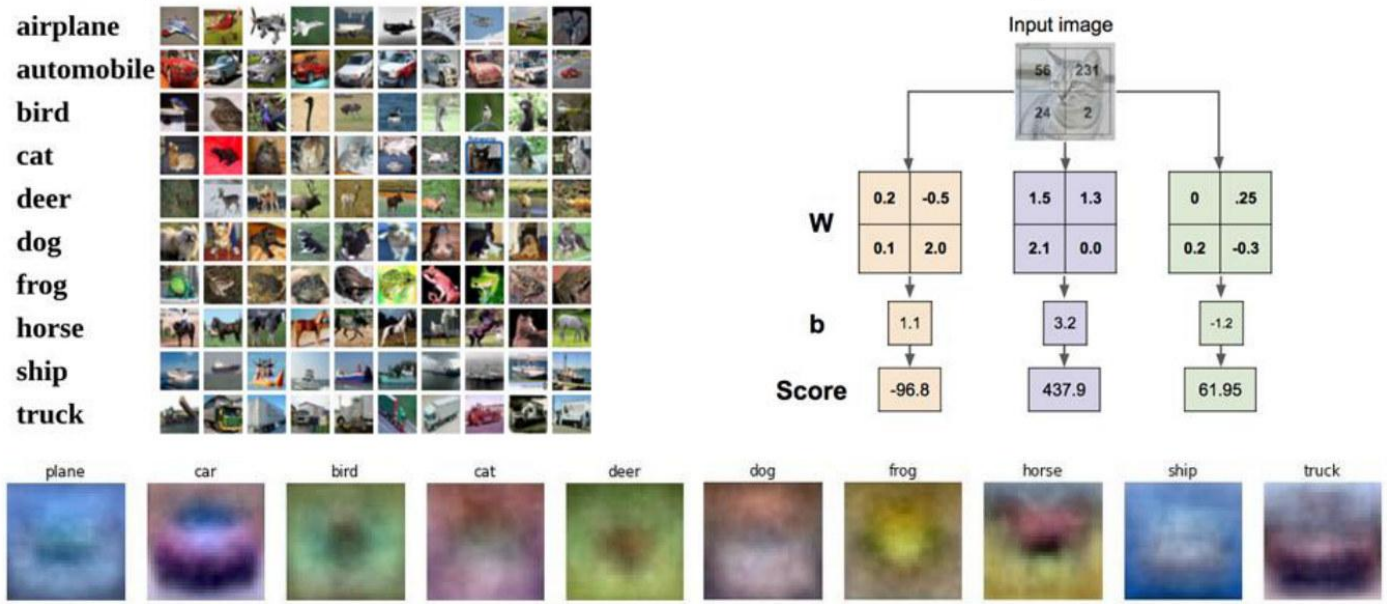




解读线性分类器：视觉视角

把 W 的每一行 reshape 回图像尺寸：
每个类别学到一张“模板”（权重模板）
分类 = 找与哪个模板最匹配

Interpreting a Linear Classifier: Visual Viewpoint



- 图：
  - 左：CIFAR-10 十个类别的训练图片网格
  - 中：4 像素小例子在三个类别下的 W、b、Score 演算
  - 下：CIFAR-10 十个类别学习到的权重模板
    （plane/car/bird/cat/deer/dog/frog/horse/ship/truck），
    模板呈现模糊的“类平均颜色”，如 ship 蓝色、horse 边缘偏红

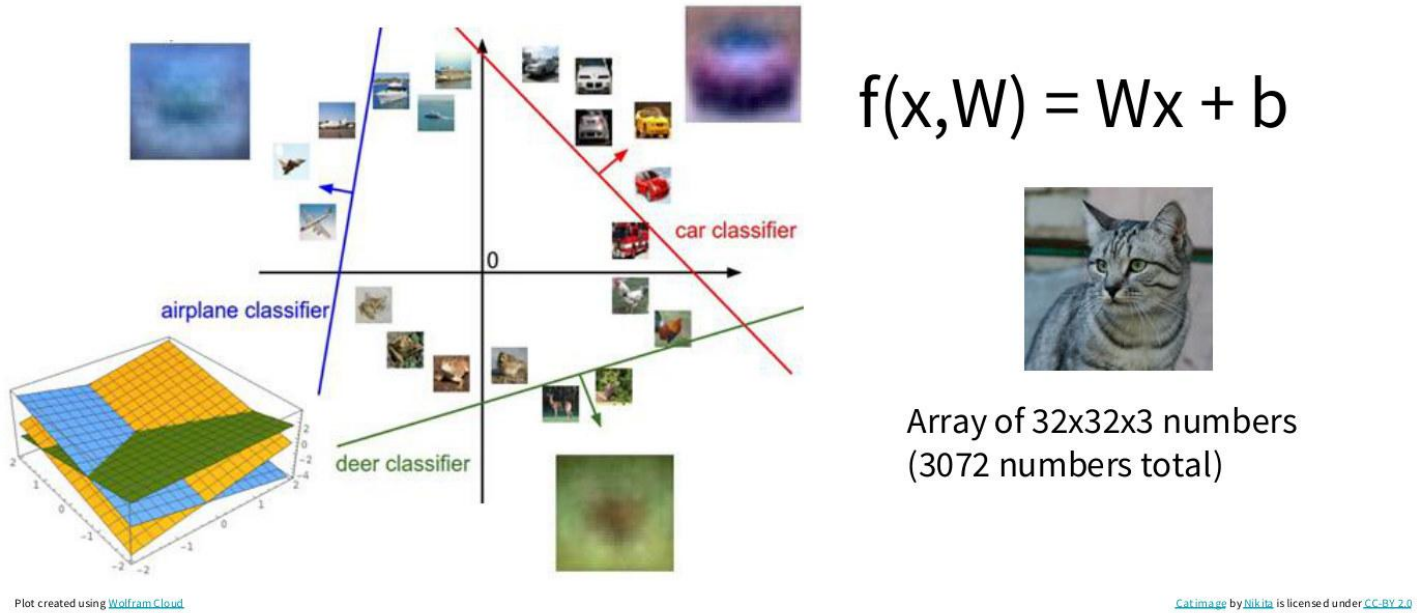
解读线性分类器：几何视角

f(x, W) = W x + b

把每张图像看作 3072 维空间中的一个点；
每个类别的得分是该空间中的线性函数：
· W 的每一行确定一个方向（超平面）
· b 决定超平面的偏移

改变 W 的一行 = 旋转对应的直线；改变 b = 平移

▸ 图：二维示意——airplane/car/deer 三个分类器各自的得分等高线方向



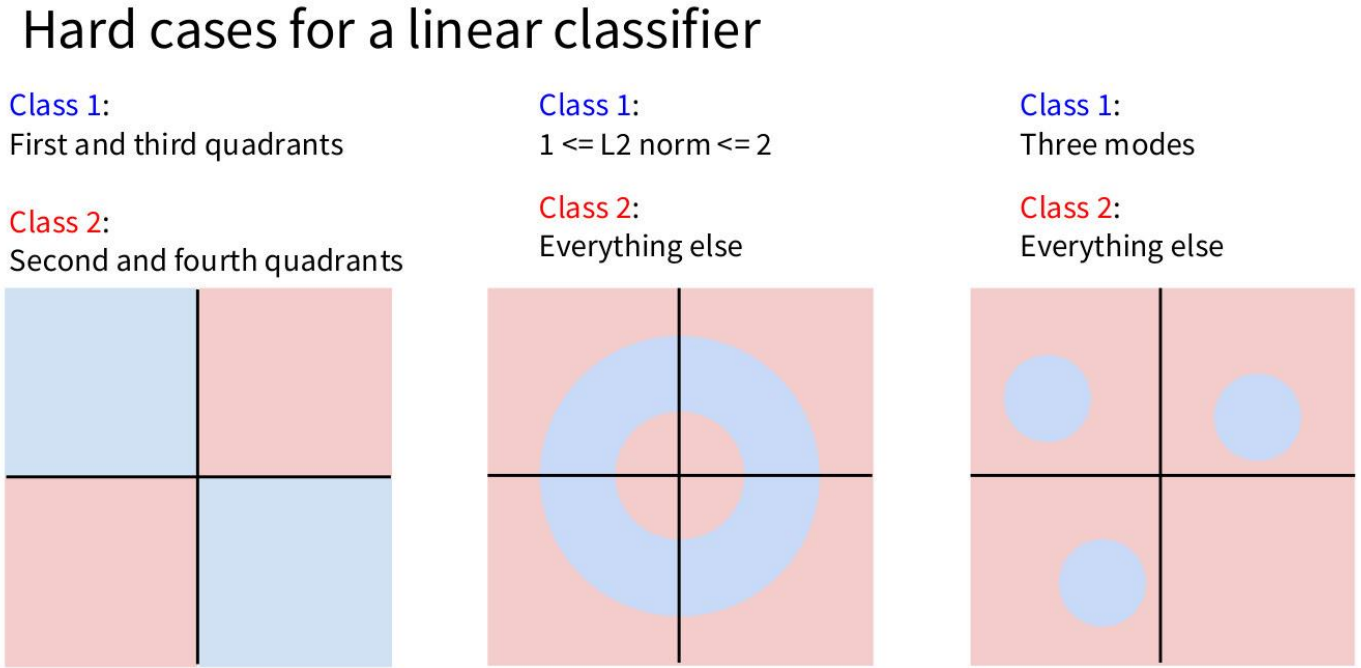
线性分类器的困难情形

线性分类器无法解决的非线性问题：

· 类 1：第一、三象限；类 2：第二、四象限（XOR 问题，一条直线分不开）
· 类 1：1 ≤ L2 范数 ≤ 2（圆环）；类 2：其余
· 类 1：三个分离的模式块；类 2：其余

▸ 图：三组二维数据分布图

→ 需要更强大的模型（神经网络）



Linear Classifier – Choose a good W



airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

Cat image by Nikita is licensed under CC-BY 2.0; Car image is CC0 1.0 public domain; Frog image is in the public domain

1. Define a loss function that quantifies our unhappiness with the scores across the training data.
2. Come up with a way of efficiently finding the parameters that minimize the loss function. (optimization)

线性分类器：如何选一个好的 W?

两步走：

1. 定义一个损失函数（loss function）：
   量化我们对“当前得分在训练数据上的表现”有多不满意

2. 想出一种高效的方法，
   找到能最小化损失函数的参数 —— 优化（optimization）

Linear Classifier – Choose a good W

$$f(x, W) = Wx$$

A loss function tells how good our current classifier is



airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

Cat image by Nikita is licensed under CC-BY 2.0; Car image is CC0 1.0 public domain; Frog image is in the public domain

损失函数：衡量分类器的好坏

损失函数告诉我们：
当前分类器到底有多好（多差）

- 图：同一张猫图，在不同 W 下
得到不同的三类得分 —— 不同的 W 好坏不同

损失函数：记号

给定样本数据集：

$\{(x_i, y_i)\}$, $i = 1 \ldots N$

其中 x_i 是图像，y_i 是（整数）标签

单样本损失：$L_i = L(f(x_i, W), y_i)$

Linear Classifier – Choose a good W

$$f(x, W) = Wx$$





airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

A loss function tells how good our current classifier is

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^{N}$$

Where x_i is image and y_i is (integer) label

Cat image by Nikita is licensed under CC-BY 2.0; Car image is CC0 1.0 public domain; Frog image is in the public domain

损失函数：数据集上的总损失

数据集上的损失 = 各样本损失的平均：

$L = (1/N) \Sigma L_i(f(x_i, W), y_i)$

接下来以 Softmax 分类器为例

Linear Classifier – Choose a good W



airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

Cat image by Nikita is licensed under CC-BY 2.0; Car image is CC0 1.0 public domain; Frog image is in the public domain

$$f(x, W) = Wx$$

A loss function tells how good our current classifier is

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^{N}$$

Where x_i is image and
y_i is (integer) label

Loss over the dataset is a average of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Softmax 分类器

Softmax 分类器（Softmax Classifier）

又称：多项逻辑回归（Multinomial Logistic Regression）

Softmax classifier

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as probabilities



cat	3.2
car	5.1
frog	-1.7

Softmax：把得分变成概率

目标：想把分类器的原始得分解读为【概率】

当前得分（logits）：
cat 3.2
frog 5.1
car -1.7

▸ 图：得分表 + “Softmax 函数”标注

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

cat	3.2
car	5.1
frog	-1.7

Softmax：引入 Softmax 函数

解决思路：引入 Softmax 函数（Softmax Function）
把原始得分变换为概率

Softmax 函数 = exp → 归一化，两步完成

▸ 图：得分表 + “Softmax Function” 标注

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as probabilities



$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

Probabilities must be >= 0

cat	3.2	exp →	24.5
car	5.1		164.0
frog	-1.7		0.18

unnormalized probabilities

Softmax：第一步 exp

对每个得分取指数（exp）：

cat: e^3.2 = 24.5

frog: e^5.1 = 164.0

car: e^-1.7 = 0.18

得到【未归一化的概率】

为什么用 exp？保证概率 ≥ 0（概率必须非负）

Softmax：第二步 归一化

除以总和进行归一化（normalize）：

总和 = 24.5 + 164.0 + 0.18 = 188.68

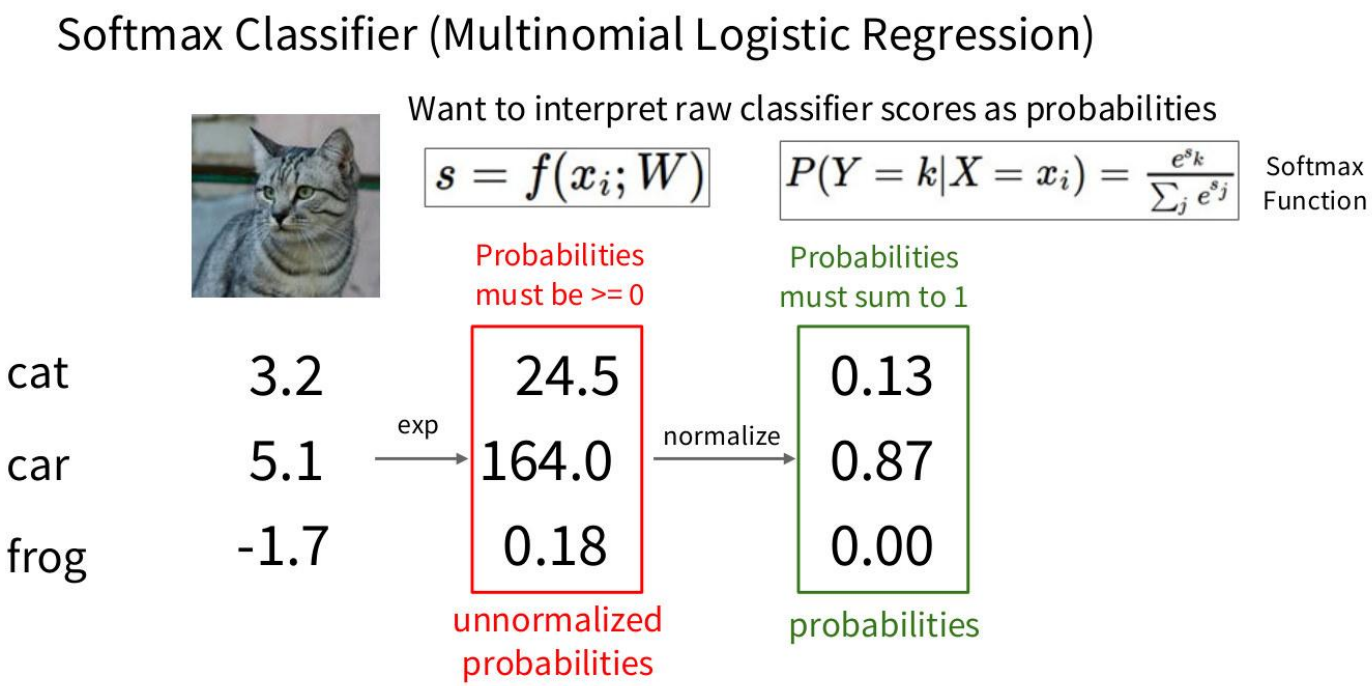
cat: 24.5 / 188.68 = 0.13

frog: 164.0 / 188.68 = 0.87

car: 0.18 / 188.68 = 0.00

得到真正的【概率】

概率之和必须等于 1



Softmax：logits 与概率

· 最终的 0.13 / 0.87 / 0.00 是概率

· 原始得分 3.2 / 5.1 / -1.7 称为：
非归一化对数概率（unnormalized log-probabilities）
或 logits

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as probabilities



$s = f(x_i; W)$

$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$ Softmax Function

Probabilities must be >= 0

Probabilities must sum to 1

		exp		normalize	
cat	3.2	→	24.5	→	0.13
car	5.1		164.0		0.87
frog	-1.7		0.18		0.00

Unnormalized log-probabilities / logits

unnormalized probabilities

probabilities

Softmax：损失

损失 = 正确类别概率的负对数：

L_i = -log(正确类的概率)
= -log(0.13)
= 2.04

正确类概率越接近 1，损失越接近 0；
越接近 0，损失趋于无穷大

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as probabilities



$s = f(x_i; W)$

$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$ Softmax Function

$L_i = -\log P(Y = y_i|X = x_i)$

	Unnormalized log-probabilities / logits	exp	unnormalized probabilities (Probabilities must be >= 0)	normalize	probabilities (Probabilities must sum to 1)	
cat	3.2		24.5		0.13	→ L_i = -log(0.13) = 2.04
car	5.1		164.0		0.87	
frog	-1.7		0.18		0.00	

Softmax：最大似然估计

最大似然估计（Maximum Likelihood Estimation, MLE）：

选择权重，使观测到的数据
在模型下的似然（likelihood）最大

（细节见 CS 229 课程）

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as probabilities



$s = f(x_i; W)$

$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$ Softmax Function

$L_i = -\log P(Y = y_i|X = x_i)$

	Unnormalized log-probabilities / logits	exp	Probabilities must be >= 0	normalize	Probabilities must sum to 1
cat	3.2		24.5		0.13
car	5.1		164.0		0.87
frog	-1.7		0.18		0.00

→ L_i = -log(0.13) = 2.04

Maximum Likelihood Estimation
Choose weights to maximize the likelihood of the observed data
(See CS 229 for details)

Softmax：与正确概率比较

把预测概率与“正确概率”比较：

真实标签 = 猫 → one-hot 目标：
cat 1.00, frog 0.00, car 0.00

预测：cat 0.13 —— 差距很大，损失大

Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as probabilities



$s = f(x_i; W)$

$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$ Softmax Function

$L_i = -\log P(Y = y_i|X = x_i)$

	Unnormalized log-probabilities / logits	exp	unnormalized probabilities (Probabilities must be >= 0)	normalize	probabilities (Probabilities must sum to 1)	compare	Correct probs
cat	3.2		24.5		0.13		1.00
car	5.1		164.0		0.87		0.00
frog	-1.7		0.18		0.00		0.00

Softmax：整体目标

合起来：

最大化【正确类别】的概率
（= 最小化负对数似然损失）

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

Maximize probability of correct class

$$L_i = -\log P(Y = y_i|X = x_i)$$

Putting it all together:

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

cat	3.2
car	5.1
frog	-1.7

Softmax：两个思考题

Q1：softmax 损失 L_i 的最小值 / 最大值可能是多少?

Q2：初始化时所有得分 s_j 近似相等；
若有 C 个类别，此时 L_i 是多少?

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

Maximize probability of correct class

$$L_i = -\log P(Y = y_i|X = x_i)$$

Putting it all together:

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

cat	3.2
car	5.1
frog	-1.7

Q1: What is the min/max possible softmax loss L_i?

Q2: At initialization all s_j will be approximately equal; what is the softmax loss L_i, assuming C classes?

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax Function

Maximize probability of correct class

$$L_i = -\log P(Y = y_i|X = x_i)$$

Putting it all together:

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

cat	3.2
car	5.1
frog	-1.7

Q2: At initialization all s will be approximately equal; what is the loss?
A: -log(1/C) = log(C),
If C = 10, then L_i = log(10) ≈ 2.3

思考题答案

Q2 答：所有得分相等时，每个类的概率 = 1/C

L_i = -log(1/C) = log(C)
若 C = 10，则 L_i = log(10) ≈ 2.3

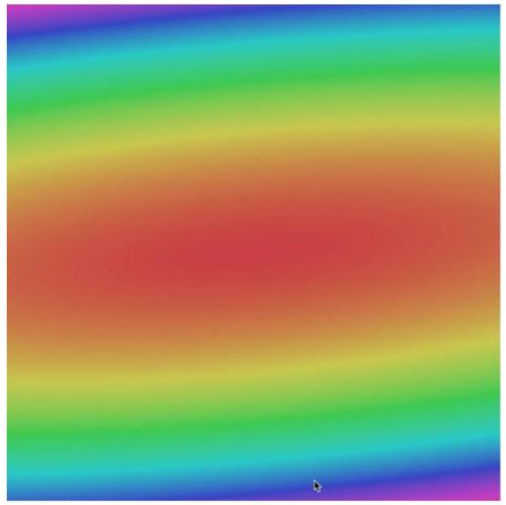
（Q1 补充：最小值趋近 0，最大值无上界）

· 训练开始时损失应约为 log(C)，
这是检查代码实现的好方法

Coming up:

- Regularization
- Optimization

$$f(x,W) = Wx + b$$



下讲预告

f(x, W) = W x + b

接下来：

· 正则化（Regularization）

· 优化（Optimization）